# EEM 364
# Mikroişlemcileri

# Assembly Dili Yapısı: Etiketler ve Döngüler

- Basit segment tanımlama ve
- EQU direktifi

```
01 .MODEL SMALL
02
03 .DATA
04
05
06 x  db 1, 2, 15, 33, 66, 5, 3, 8, 9, 25, 34, 22
07 lx equ $-x
08 ts db 25
09
10
11 U db ?
12 V db ?
13
14 .CODE
15
16 mov ax , @data
17 mov ds , ax
18
```

# Koşullar ve Döngüler

- JNZ/JZ
- INC
- DEC
- OFFSET
- EQU
- LEA
- CMP
- JE , JG , JL

```
CMP register, register;
CMP AX, BX
CMP register, memory;
CMP AX, SONUC
CMP register,
  immediate;
CMP AX,5
```

```
CMP memory, register;
CMP SONUC, AX
CMP memory, immediate;
CMP SONUC,5
```

https://web.karabuk.edu.tr/emelkocak/indir/MTM305/Kontrol_Komutlar%C4%B1.pdf

# Assembly Dili Yap

## Koşul Kurma

Belli bir sayıdan küçük olanları toplamak

```
.MODEL SMALL

.DATA


x  db 1, 2, 15, 33, 66, 5, 3, 8, 9, 25, 34, 22
lx equ $-x
ts db 10

TP db 0

.CODE

mov ax , @data
mov ds , ax



mov si , 0
mov cx , lx
mov al , 0
mov ah , 0

KAR: mov bl , x[si]
     cmp bl , ts
     jl TPL
     inc si
     dec cx
     jnz KAR



SON:
mov TP , al



hlt

TPL:  add al , bl
      inc si
      dec cx
      jz  SON
      jmp KAR
```

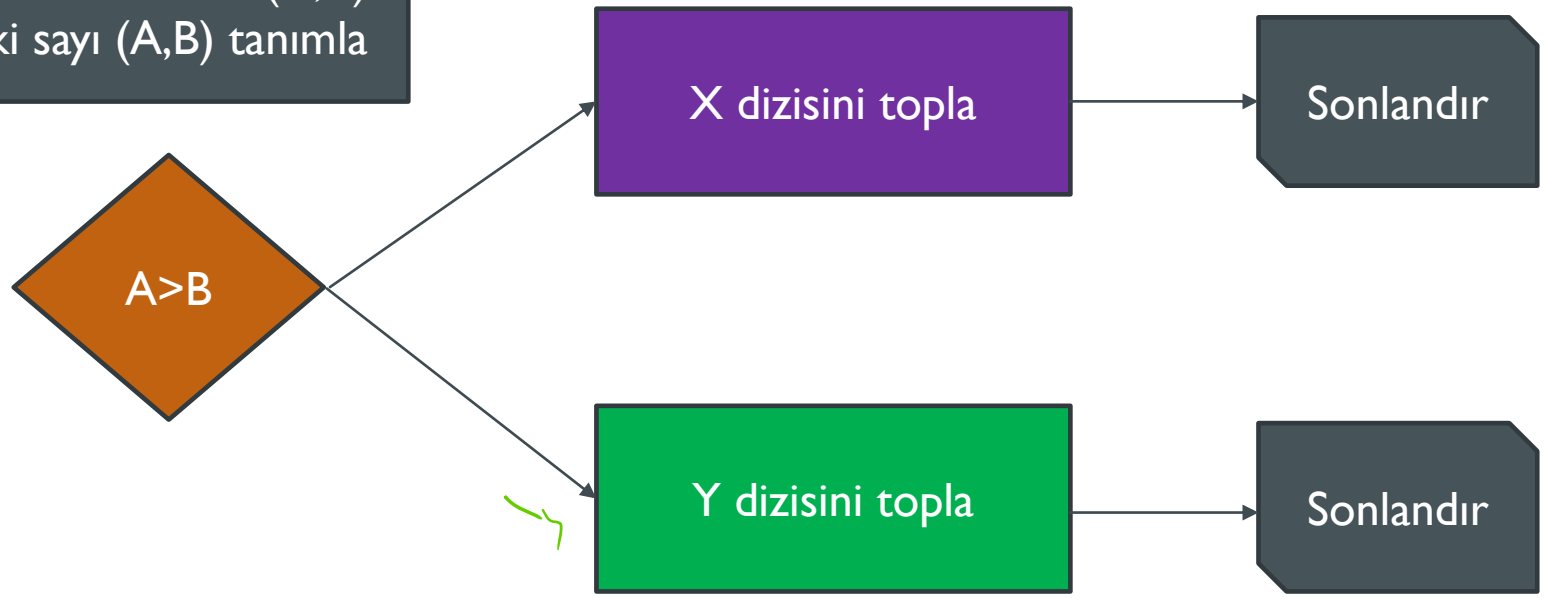
# Koşullar

Dizi içindeki elemanlar belli bir sayıdan büyükse U'da değilse V'de toplansın

```
.MODEL SMALL

.DATA


x  db 1, 2, 15, 33, 66, 5, 3, 8, 9, 25, 34, 22
lx equ $-x
ts db 25


U db ?
V db ?

.CODE

mov ax , @data
mov ds , ax

mov si , 0
mov cx , lx
mov al , 0
mov ah , 0


KAR: mov bl , x[si]
     cmp bl , ts
     jl VL
     jg UL


SON:
mov V , al
mov U , ah


hlt

VL:  add al , bl
     inc si
     dec cx
     jz  SON
     jmp KAR

UL:  add ah , bl
     inc si
     dec cx
     jz  SON
     jmp KAR
```

# Koşullar ve Döngüler

İki dizi tanımla (X,Y)
İki sayı (A,B) tanımla




```
if A>B
    for i in range(x)
        T+=i

else
    for i in range(y)
        Z+=i
```

# Koşullar ve Döngüler

- JNZ/JZ
- CMP
- INC
- DEC
- OFFSET
- EQU
- LEA
- CMP
- JE , JG , JL
- TEST

## Bit Tabanlı AND İşlemi

```
.MODEL SMALL

.DATA

.CODE

mov ax , @data
mov ds , ax


mov al , 5
mov bl , 8

test al , 01H
test bl , 01H


hlt
```

# Koşullar ve Döngüler

- JNZ/JZ
- CMP
- INC
- DEC
- OFFSET
- EQU
- LEA
- CMP
- JE , JG , JL
- TEST

## Bit Tabanlı AND İşlemi

```
.MODEL SMALL

.DATA

.CODE

mov ax , @data
mov ds , ax


mov al , 5
mov bl , 8

test al , 01H
test bl , 01H


hlt
```

Sayıları çift ve tek olma durumuna göre toplama

# Koşullar ve Döngüler ve Prosedürler !!

- JNZ/JZ
- CMP
- INC
- DEC
- OFFSET
- EQU
- LEA
- CMP
- JE , JG , JL
- TEST
- PROC

```
.MODEL SMALL

.DATA
.CODE


main proc far
        mov ax , @data
        mov ds , ax

        call F1

        mov ax , bx


        hlt
main endp


F1 PROC
       mov bx , 1233H
       ret
F1 ENDP
```

# Koşullar ve Döngüler ve Prosedürler !!

- JNZ/JZ
- CMP
- INC
- DEC
- OFFSET
- EQU
- LEA
- CMP
- JE , JG , JL
- TEST
- PROC
- LOOP

```
.MODEL SMALL

.DATA
.CODE


main proc far
        mov ax , @data
        mov ds , ax

        call F1

        mov ax , bx


        hlt
main endp


F1 PROC
       mov bx , 1233H
       ret
F1 ENDP
```

Loop Komutu İle Toplama

- JNZ/JZ
- CMP
- INC
- DEC
- OFFSET
- EQU
- LEA
- CMP
- JE , JG , JL
- TEST
- PROC
- LOOP

```
.MODEL SMALL


.DATA
    x  db 1, 2, 15, 33, 66, 5, 3, 8, 9, 25, 34, 22
    lx equ $-x
    T db ?

.CODE


main proc far

     mov ax , @data
     mov ds , ax

     call F1

     mov T , al

     hlt

main endp


F1 proc

    mov cx , lx
    mov al , 0

    lea si , x

    L:
      add al , [si]
      inc si
      loop L

    ret

F1 endp
```

# LAB ÖDEVİ-1

x db 1,2,4,5,7,1 y db 3,1,5,2,2,4 iki tane dizi olsun |x(i) - y(i)| değerlerini bulunuz. Bunları M sembolüne atayınız.

İpucu: Mutlak değer işlemi asm'de bulunmamaktadır. Sayıları büyük ve küçük olarak karşılaştırıp her zaman büyük olandan küçük olanı çıkaracak şekilde bir koşul yazmanız gerekecektir.

# LAB ÖDEVİ-2

x db 1,2,4,5,7,1

Sayılarını toplayan prosedürü yazınız. Sonucu T sembolüne yazdırınız.

# Koşullar ve Döngüler ve Prosedürler

## Sıralama algoritması

```
.MODEL SMALL

.DATA

x    db 1,3,5,2,2,1,8,0,4, 7
lx equ $-x-1

.CODE

main proc far
     mov ax , @data
     mov ds , ax

L0:
     mov ax , 0
     mov cx , lx
     mov si , 0
     mov dl , 0



L:
     mov al , x[si]
     mov ah , x[si+1]

     call SWP
     inc si
     loop L

     cmp dl , 0

     jnz L0

HLT

main endp

SWP proc

     cmp al , ah
     jle not_swp
     mov x[si+1] , al
     mov x[si]   , ah

     inc dl

     not_swp:
     ret
SWP endp
```

# Koşullar ve Döngüler ve

## Çarpma ve Bölme

```
.MODEL SMALL

.DATA

A equ  15
C equ  4

Z dw 0


.CODE

main proc far

    mov ax , @data
    mov ds , ax


    mov ax , 0
    mov bx , 0

    call carp

    mov Z , bx

    HLT

main endp


carp proc

    mov al , A
    mov cx , C


    CAR: add bx , ax
         loop CAR


    ret

carp endp
```

```
.MODEL SMALL

.DATA

X equ  53
B equ  7


Z dw 0


.CODE

main proc far

    mov ax , @data
    mov ds , ax

    mov ax , 0

    call bol

    hlt

main endp


bol proc

    mov al , X
    mov bx , 0

    L:   inc ah
         sub al , B
         cmp al , B
         jge L
    ret

bol endp
```

# Koşullar ve Döngüler ve Prosedürler !!

- **JNZ/JZ**
- **CMP**
- **INC**
- **DEC**
- **OFFSET**
- **EQU**
- **LEA**

- **CMP**
- **JE , JG , JL**
- **TEST**
- **PROC**
- **LOOP**
- **MUL**

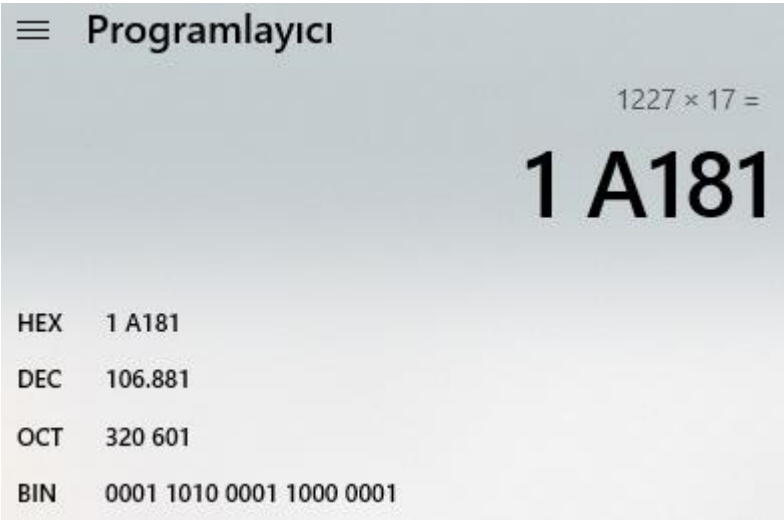
- **MUL ve IMUL**
- reg , reg
- reg , mem

- **Çarpılan**:
  - AL veya AX
- **Çarpan**:
  - BL,CL,DL
  - BX,CX,DX

# Prosedürler Devam

- **MUL ve IMUL**
- reg , reg
- reg , mem

```
MOV AX , 1227H
MOV DX , 17H
MUL DX
```




- **Çarpılan**:
  - AL veya AX
- **Çarpan**:
  - BL,CL,DL
  - BX,CX,DX

| YAZMAÇLAR | |
|---|---|
| AH: 12 | AL : 27 |
| AX :A1 81 | |
| DX: 00 01 | |

# Prosedürler Devam

- **MUL ve IMUL**
- reg , reg
- reg , mem

- **Çarpılan**:
  - AL veya AX
- **Çarpan**:
  - BL,CL,DL
  - BX,CX,DX

```
MOV AX , 1227H
MOV DX , 17H
MUL DX
```

```
MOV AX , 1227H
MOV DL , 17H
MUL DL
```

```
01 mov ax , 1203H
02
03 mov dl , 2
04
05 mul dl
06
07
```

file math debug

Load

registers

| | H | L |
|---|---|---|
| AX | 00 | 06 |
| BX | 00 | 00 |
| CX | 00 | 00 |

Programlayıcı

1227 × 17 =

1 A181

| | |
|---|---|
| HEX | 1 A181 |
| DEC | 106.881 |
| OCT | 320 601 |
| BIN | 0001 1010 0001 1000 0001 |

| YAZMAÇLAR | |
|---|---|
| AH: 12 | AL : 27 |
| AX :A1 81 | |
| DX: 00 01 | |

# Prosedürler Devam

- **MUL ve IMUL**
- reg , reg
- reg , mem

- **Çarpılan:**
  - AL veya AX
- **Çarpan:**
  - BL,CL,DL
  - BX,CX,DX

```
01 T dw 0
02
03 mov al , 180
04
05 mov dl , 2
06
07 mul dl
08
09 mov T , ax
10
```

size: word  elements: 1
edit  show as: unsigned

T  360

registers

| | H | L |
|---|---|---|
| AX | 01 | 68 |
| BX | 00 | 00 |
| CX | 00 | 00 |
| DX | 00 | 02 |
| CS | 0100 | |
| IP | 000B | |

# Prosedürler Devam

- **MUL ve IMUL**
- reg , reg
- reg , mem

- **Çarpılan**:
  - AL veya AX
- **Çarpan**:
  - BL,CL,DL
  - BX,CX,DX

```
.MODEL SMALL

.DATA

X equ  6

S dw 0


.CODE

mov ax , @data
mov ds , ax


mov ax , 0001H
mov cx , X

L:
mul cx
loop L

mov S , ax


HLT
```

**Faktöriyel !**

# Prosedürler Devam

- **DIV ve IDIV**
- reg , reg
- reg , mem

```
MOV AL , 27H ; 39
MOV DL , 17H ; 23
DIV DL
```

| YAZMAÇLAR | |
|---|---|
| AH: ~ | AL : 27 |
| AX: 10 01 | |

- **Bölünen**:
  - AL veya AX
- **Bölen**:
  - BL, CL, DL
  - BX, CX, DX

# Prosedürler Devam

- **DIV ve IDIV**
- reg , reg
- reg , mem

```
MOV AL , 27H ; 39
MOV DL , 17H ; 23
DIV DL
```

| YAZMAÇLAR | |
|---|---|
| AH: ~ | AL : 27 |
| AX: 10 01 | |

- **Bölünen**:
  - AL veya AX
- **Bölen**:
  - BL, CL, DL
  - BX, CX, DX

```
MOV AL , 42H; 66
MOV DL , 12H; 18
DIV DL
```

| YAZMAÇLAR | |
|---|---|
| AH: ~ | AL : 42 |
| AX: 0C 03 | |

# Prosedürler Devam

- JNZ/JZ
- CMP
- INC
- DEC
- OFFSET
- EQU
- LEA

- CMP
- JE , JG , JL , JNS
- TEST
- PROC
- LOOP
- MUL
- DIV

**Asal Sayı Bulma**

```
.MODEL SMALL

.DATA
x equ 53
y equ x/2

BN db  0
BM db 0
FL db 1

.CODE

main proc far

mov ax , @Data
mov ds , ax

mov cl , 2
mov dx , x

call asal


hlt
main endp

asal proc

    L:
    mov ax , dx
    div cl
    cmp ah , 0
    jz not_asal
    inc cl
    cmp cl , y
    jnz L


    D:

    ret

    not_asal:

    mov BN , cl
    mov BM , al
    mov FL , 0

    jmp D

asal endp
```

# İç İçe Döngüler

- JNZ/JZ
- CMP
- INC
- DEC
- OFFSET
- EQU
- LEA

- CMP
- JE , JG , JL , JNS
- TEST
- PROC
- LOOP
- MUL
- DIV
- PUSH
- POP

# İç İçe Döngüler

- JNZ/JZ
- CMP
- INC
- DEC
- OFFSET
- EQU
- LEA

- CMP
- JE , JG , JL , JNS
- TEST
- PROC
- LOOP
- MUL
- DIV
- PUSH
- POP

**Matris Satır Toplama**

```
.MODEL SMALL

.DATA


 x   db 1,2,3,4 , 5,6,7,8 , 9,10,11,12
 lx  equ 3
 ly  equ 4

 T db lx dup(0)

.CODE

Main PROC Far

    mov ax , @data
    mov ds , ax



    mov ax , 0
    mov cx , lx

    mov si , 0
    mov di , 0

    L1: push cx
        mov cx , ly
        mov dl , 0

        L2:
            mov al , x[si]
            add dl , al
            inc si
        loop L2

        mov T[di] , dl
        inc di

        pop cx

    loop L1


HLT

Main ENDP
```

```
.MODEL SMALL
.DATA

 x   db 1,2,3,4 , 5,6,7,8 , 9,10,11,12
 lx  equ 3
 ly  equ 4

 T  db lx dup(0)
```

# İç İçe Döngüler

# İç İçe Döngüler

```
.MODEL SMALL
.DATA

 x  db 1,2,3,4 , 5,6,7,8 , 9,10,11,12
 lx equ 3
 ly equ 4

 T db lx dup(0)
.CODE
Main PROC Far

    mov ax , @data
    mov ds , ax
```

```
HLT
Main ENDP
```

# İç İçe Döngüler

```
.MODEL SMALL
.DATA

 x   db 1,2,3,4 , 5,6,7,8 , 9,10,11,12
 lx  equ 3
 ly  equ 4

 T db lx dup(0)
.CODE
Main PROC Far

    mov ax , @data
    mov ds , ax

        mov al , x[si]
        add dl , al

HLT
Main ENDP
```

# İç İçe Döngüler

```
.MODEL SMALL
.DATA

 x  db 1,2,3,4 , 5,6,7,8 , 9,10,11,12
 lx equ 3
 ly equ 4

 T db lx dup(0)
.CODE
Main PROC Far

    mov ax , @data
    mov ds , ax
```

```
        L2:
            mov al , x[si]
            add dl , al
            inc si
        loop L2

HLT
Main ENDP
```

# İç İçe Döngüler

```
.MODEL SMALL
.DATA

 x  db 1,2,3,4 , 5,6,7,8 , 9,10,11,12
 lx equ 3
 ly equ 4

 T db lx dup(0)
.CODE
Main PROC Far

    mov ax , @data
    mov ds , ax

        mov cx , ly
        mov dl , 0

        L2:
            mov al , x[si]
            add dl , al
            inc si
        loop L2

        mov T[di] , dl
        inc di

HLT
Main ENDP
```

# İç İçe Döngüler

```
.MODEL SMALL
.DATA

 x  db 1,2,3,4 , 5,6,7,8 , 9,10,11,12
 lx equ 3
 ly equ 4

 T db lx dup(0)
.CODE
Main PROC Far

    mov ax , @data
    mov ds , ax


    mov ax , 0
    mov cx , lx

    mov si , 0
    mov di , 0

    L1: push cx
        mov cx , ly
        mov dl , 0

        L2:
            mov al , x[si]
            add dl , al
            inc si
        loop L2

        mov T[di] , dl
        inc di

        pop cx

    loop L1

HLT
Main ENDP
```

# İç İçe Döngüler

- JNZ/JZ
- CMP
- INC
- DEC
- OFFSET
- EQU
- LEA

- CMP
- JE , JG , JL , JNS
- TEST
- PROC
- LOOP
- MUL
- DIV
- PUSH
- POP

**Matris içindeki bir elemanın indisini bulma**

```
.MODEL SMALL

.DATA

 x  db 1,2,3,4,  5,6,7,8,  9,10,11,12
 lx equ $-x

.CODE


Main PROC FAR

      mov ax , @data
      mov ds , ax


      mov dh , 0

      L1:


         mov dl , 0

         L2:
             mov al , x[si]
             cmp x[si] , 7
             je K

             inc si


             inc dl
             cmp dl , 4
             jne L2


       inc dh
       cmp dh , 3
       jne L1


      K:
      hlt


Main ENDP
```

# Kesmeler

## DOS INT 21h - DOS Function Codes

The follow abridged list of DOS interrupts has been extracted from a large list compile

| AH | Description | AH | Description |
|---|---|---|---|
| 01 | Read character from STDIN | 02 | Write character to STDOUT |
| 05 | Write character to printer | 06 | Console Input/Output |
| 07 | Direct char read (STDIN), no echo | 08 | Char read from STDIN, no echo |
| 09 | Write string to STDOUT | 0A | Buffered input |
| 0B | Get STDIN status | 0C | Flush buffer for STDIN |
| 0D | Disk reset | 0E | Select default drive |
| 19 | Get current default drive | 25 | Set interrupt vector |
| 2A | Get system date | 2B | Set system date |
| 2C | Get system time | 2D | Set system time |
| 2E | Set verify flag | 30 | Get DOS version |
| 35 | Get Interrupt vector | | |
| 36 | Get free disk space | 39 | Create subdirectory |
| 3A | Remove subdirectory | 3B | Set working directory |
| 3C | Create file | 3D | Open file |
| 3E | Close file | 3F | Read file |
| 40 | Write file | 41 | Delete file |
| 42 | Seek file | 43 | Get/Set file attributes |
| 47 | Get current directory | 4C | Exit program |
| 4D | Get return code | 54 | Get verify flag |
| 56 | Rename file | 57 | Get/Set file date |

Yazılım kesmeleri INT makine komutuyla oluşturulan kesmelerdir. Programcı kesme oluşturmak istediği zaman INT makine komutunu kullanabilir. INT makine komutu tıpkı CALL makine komutu gibi program akışını kesme koduna geçirmektedir. INT makine komutuyla oluşturulan kesmeden IRET makine komutuyla geri dönülür.

Donanım kesmeleri mikroişlemcinin dışındaki bir birim tarafından mikroişlemcinin INT ucunun aktive edilmesiyle oluşturulur. Kesme denilince ilk akla gelen donanım kesmeleridir. Mikroişlemci bir makine komutunu çalıştırdıktan sonra, bir sonraki makine

http://www.mentis.com.tr/web/80x86-assembly-version1.pdf

# Kesmeler

## DOS INT 21h - DOS Function Codes

The follow abridged list of DOS interrupts has been extracted from a large list compile

| AH | Description | AH | Description |
|---|---|---|---|
| 01 | Read character from STDIN | 02 | Write character to STDOUT |
| 05 | Write character to printer | 06 | Console Input/Output |
| 07 | Direct char read (STDIN), no echo | 08 | Char read from STDIN, no echo |
| 09 | Write string to STDOUT | 0A | Buffered input |
| 0B | Get STDIN status | 0C | Flush buffer for STDIN |
| 0D | Disk reset | 0E | Select default drive |
| 19 | Get current default drive | 25 | Set interrupt vector |
| 2A | Get system date | 2B | Set system date |
| 2C | Get system time | 2D | Set system time |
| 2E | Set verify flag | 30 | Get DOS version |
| 35 | Get Interrupt vector | | |
| 36 | Get free disk space | 39 | Create subdirectory |
| 3A | Remove subdirectory | 3B | Set working directory |
| 3C | Create file | 3D | Open file |
| 3E | Close file | 3F | Read file |
| 40 | Write file | 41 | Delete file |
| 42 | Seek file | 43 | Get/Set file attributes |
| 47 | Get current directory | 4C | Exit program |
| 4D | Get return code | 54 | Get verify flag |
| 56 | Rename file | 57 | Get/Set file date |

```
.model small
.data
    msg db "Hello, World! $"
.code
main proc far
    mov ax , @data
    mov ds ,ax

    mov  dx, offset msg
    mov  ah, 9
    int  21H

    mov  ah, 4CH
    int  21H

main endp
```

# Kesmeler

## DOS INT 21h - DOS Function Codes

The follow abridged list of DOS interrupts has been extracted from a large list compile

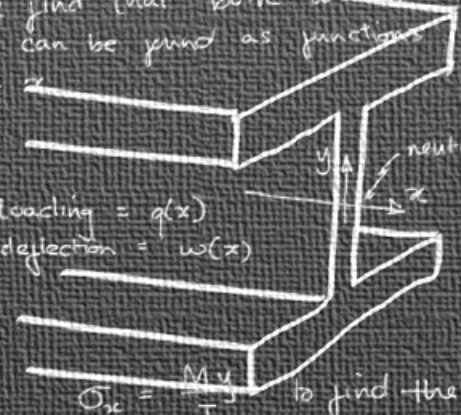
| AH | Description | AH | Description |
|---|---|---|---|
| 01 | Read character from STDIN | 02 | Write character to STDOUT |
| 05 | Write character to printer | 06 | Console Input/Output |
| 07 | Direct char read (STDIN), no echo | 08 | Char read from STDIN, no echo |
| 09 | Write string to STDOUT | 0A | Buffered input |
| 0B | Get STDIN status | 0C | Flush buffer for STDIN |
| 0D | Disk reset | 0E | Select default drive |
| 19 | Get current default drive | 25 | Set interrupt vector |
| 2A | Get system date | 2B | Set system date |
| 2C | Get system time | 2D | Set system time |
| 2E | Set verify flag | 30 | Get DOS version |
| 35 | Get Interrupt vector | | |
| 36 | Get free disk space | 39 | Create subdirectory |
| 3A | Remove subdirectory | 3B | Set working directory |
| 3C | Create file | 3D | Open file |
| 3E | Close file | 3F | Read file |
| 40 | Write file | 41 | Delete file |
| 42 | Seek file | 43 | Get/Set file attributes |
| 47 | Get current directory | 4C | Exit program |
| 4D | Get return code | 54 | Get verify flag |
| 56 | Rename file | 57 | Get/Set file date |

```
.model small

.data

    msg db "Bir sayi giriniz:  $"

.code

main proc far
    mov ax , @data
    mov ds ,ax

    mov dx, offset msg

    mov  ah, 9
    int  21H


    mov ah , 01H
    int  21H


    mov  ah, 4CH   ; HLT YERINE
    int  21H
main endp
```

# LAB ÖDEVİ

- **Bir dizindeki max elemanı bulma**
- **Palindrom: 12321 veya KABAK**
- **Bir dizinin ortalamasını bulmak**

given a beam of constant EI
we find that both w and
q can be found as functions
of x




loading = $q(x)$
deflection = $w(x)$

$\sigma_x = \frac{My}{I}$ to find the tension force at some location y

for a static system

$$\frac{\partial^2}{\partial x^2}\left(EI\frac{\partial^2 w}{\partial x^2}\right) = q$$

for $\frac{\partial EI(x)}{\partial x} = 0$

where $\frac{\partial^4 w}{\partial x^4} EI = q(x)$

E = Young's Modulus

I = Second moment of area

http://bsnscb.com/data/out/124/39800806-maths-wallpapers.jpg